Donde está en la teoría cada ejercicio:

Test 1 (Entidades y Repositorios): T4 (págs. 45-60, mapeo de entidades) y T5 (págs. 12-15, CrudRepository y págs. 77-81, validaciones JSR-303).

Test 2 (Relaciones JPA): T4 (págs. 50-54 y 88-90, @ManyToOne, @ManyToMany, cardinalidades) y T5 (págs. 61-64 y 79, @Size en colecciones).

Tests 3 y 4 (data.sql): T1 (pág. 74 y 90) y T4 (págs. 88-90 sobre cómo se traducen las claves foráneas y tablas intermedias al modelo relacional RM).

Test 5 (Servicios y @Transactional): T4 (págs. 17-20) y T5 (págs. 17, 27-47 y 84-86, transacciones, rollbackFor y validación semántica con excepciones).

Test 6 (Consultas @Query JPQL): T5 (págs. 16-22, JPQL). NO ENTRA

Test 7 (Controlador REST, Seguridad y Excepciones): T1 (págs. 49, 84), T3 (págs. 58-69, @GetMapping, @PathVariable, códigos HTTP y excepciones) y T6 (págs. 13-16).

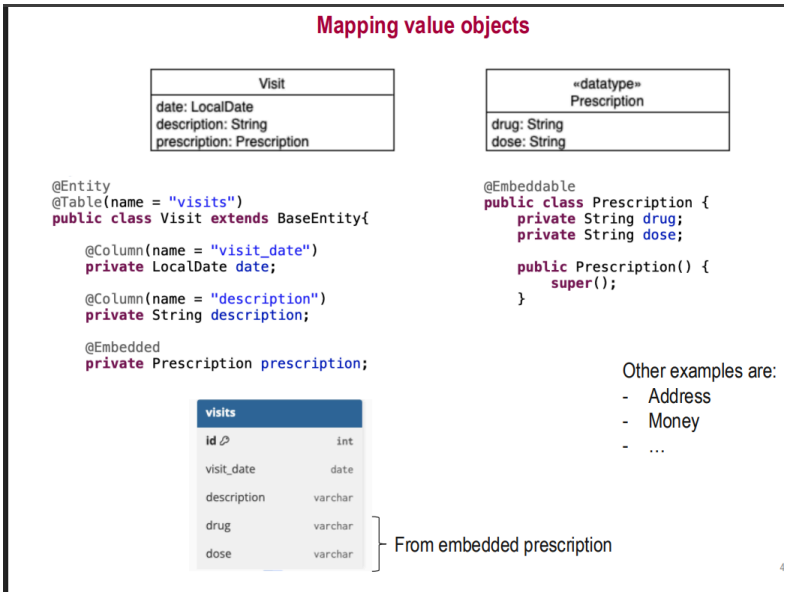
Test 8 (Pruebas unitarias JUnit): T1 (pág. 3, introducción a SWEBOK Testing) y T3 (pág. 31, JUnit), aunque el grueso de pruebas unitarias de backend se imparte en los temas de Testing posteriores al T7. NO ENTRA

Tests 9 y 10 (Componentes React, fetch, estado y props): T1 (págs. 47-48), T2 (págs. 24-40 y 55-56) y T6 (págs. 54-65).

CONCEPTOS TEORÍA RELACIONADOS CON EL TEST 1:

Para el mapeo de objetos de valor: @Embeddable: indica que sus atributos se integrarán directamente dentro de la tabla de la entidad que la contenga.

@Embedded: ponernos el atributo en la entidad.



@Digits(fraction = 0, integer = 10): restringe el teléfono a un valor numérico de hasta 10 dígitos sin decimales.



@NotNull:

- **Regla:** El elemento es válido siempre que no sea null, **pero puede estar vacío**.

@NotEmpty:

- **Regla:** Es válido siempre que no sea null y su tamaño o longitud sea estrictamente mayor que cero (size/length > 0).

@NotBlank:

- **Regla:** Es específico para cadenas de texto (CharSequence / String); es válido si no es null y su longitud tras eliminar espacios en blanco en los extremos (*trimmed*) es mayor que cero. No acepta cadenas formadas únicamente por espacios en blanco (como " "). Se usa por ejemplo con los emails (@Email)

@Enumerated(EnumType.STRING)

@Transient: Para indicar que un atributo es volátil (no persista en la base de datos).

EJEMPLOS DE RESTRICCIONES:

federationLicenseCode: obligatorio, exactamente 8 caracteres alfanuméricos que empiecen por "FED-" seguidos de 4 dígitos:

- @NotBlank

  @Pattern(regexp = "^FED-\\d{4}$")

  private String federationLicenseCode;

entryFee: número decimal obligatorio entre 5.0 y 150.0 euros, con un máximo de 2 decimales.

- @NotNull

  @DecimalMin("5.0")

  @DecimalMax("150.0")

  @Digits(integer = 3, fraction = 2)

  private BigDecimal entryFee;

roomName: obligatorio, tamaño entre 3 y 30 caracteres, y debe ser único en la tabla de la BD.

- @NotBlank

  @Size(min = 3, max = 30)

  @Column(unique = true)

  private String roomName;

sessionDate: fecha y hora obligatoria que debe pertenecer al futuro.

- @NotNull

  @Future

  private LocalDateTime sessionDate;

Restricción integer:

  @Min(1)

  @Max(15)

  Integer expectedMoveCount;

Un texto (String) supera los 255 caracteres:

@Size(max=4000)

@Column(length = 4000)

String notes;

Integer toma valor entre 1 y 8:

@NotNull

@Range (min = 1, max = 8)

Integer originX;

Si te aparece el Integer Id para no usar el BaseEntity:

  @Id

  @GeneratedValue(strategy = GenerationType.IDENTITY)

  Integer id;

- El atributo de tipo fecha (LocalDate) llamado “registeredOn”, que representa la fecha en que el alumno se registra en el curso, seguirá el formato “dd/MM/yyyy”. Este atributo debe ser obligatorio. En la base de datos se almacenará con el nombre de columna “registrationDate”.

```
@Getter
@Setter
@Entity
public class CoursePayment extends BaseEntity {
    @DateTimeFormat(pattern = "dd/MM/yyy")
    @NotNull
    @Column(name = "paymentDate")
    LocalDate paidOn;
    @NotNull
    Double amount;
}
```

CONCEPTOS DE TEORÍA RELACIONADOS CON EL TEST 2:

JAVA

```java
@Entity
public class A {
   …
   @ManyToOne(optional=false)
   private B b;
   …
}

@Entity
public class B {
   // No reference to class A
   …
}
```

Specifies if b can be null or not

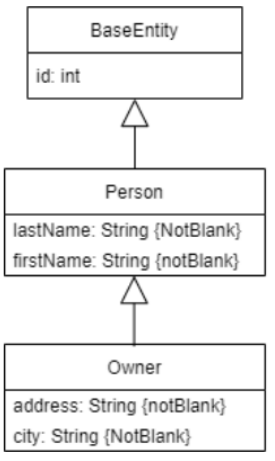
Specifies the cardinality of the association. It can be:
- @ManyToOne (A has one B, B could have multiple A)
- @OneToOne (A has one B, B has one A)
- @OneToMany (A has multiple B, B has one A)
- @ManyToMany (A has multiple B, B has multiple A)

Como saber si tengo que poner “mappedBy”:

- Si la relación es **unidireccional** (solo una clase conoce a la otra): **Nunca** se pone mappedBy.
- Si la relación es **bidireccional** (ambas clases se conocen): Tienes que poner mappedBy exactamente en el lado del **@OneToMany** (o en el lado inverso del @ManyToMany), indicando entre comillas el nombre exacto del atributo que está en la otra clase (por ejemplo, mappedBy = "owner").

HERENCIAS:

UML



JAVA

```java
@MappedSuperclass
public class BaseEntity {
   @Id
   @GeneratedValue(strategy = GenerationType.IDENTITY)
   protected Integer id;
   …
}

@MappedSuperclass
public class Person extends BaseEntity {
   @Column(name = "first_name")
   @NotEmpty
   protected String firstName;

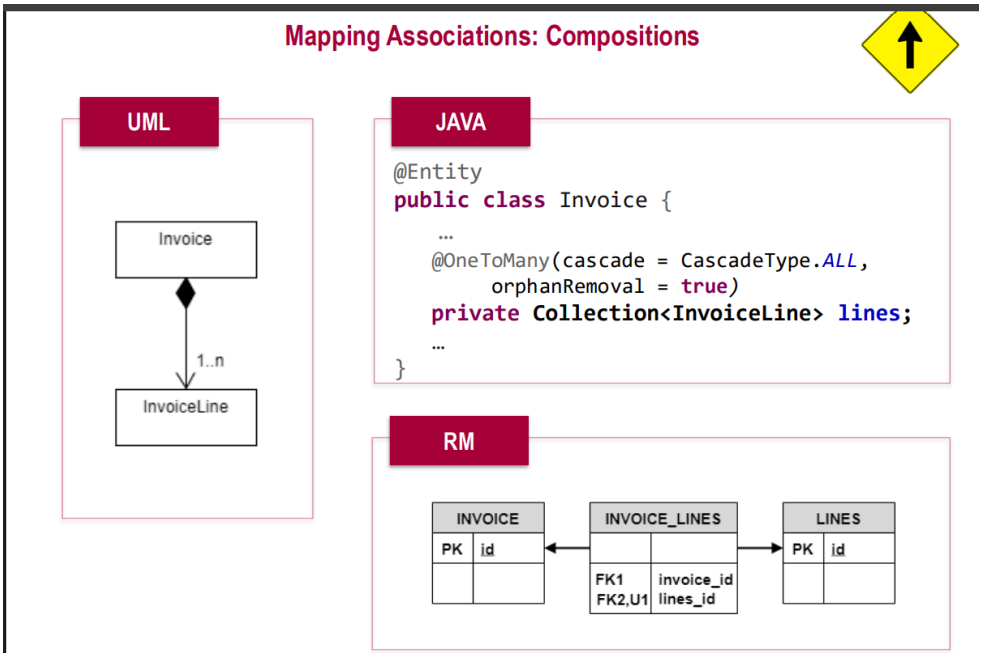
   @Column(name = "last_name")
   @NotEmpty
   protected String lastName;
   …
}

@Entity
public class Owner extends Person {
   private String address;
   private String city;
   private String telephone;
   …
}
```

55



RELACIÓN COMPOSICIÓN:



1. Relaciones @ManyToOne (Muchos a Uno)

- **Obligatorio siempre:** La anotación `@ManyToOne`.
- **Obligatorio si no puede ser nulo (Cardinalidad 1):** Debes añadir `optional = false` dentro de la anotación o usar `@NotNull`.
- *Práctica recomendada:* `@JoinColumn(name = "columna_id")` para controlar exactamente cómo se llama la clave foránea en la base de datos (es lo que usamos en el Test 2 para cuadrar con tu script SQL).

2. Relaciones @OneToMany (Uno a Muchos)

- **Obligatorio siempre:** La anotación `@OneToMany` sobre una colección (como `List<Entidad>` o `Set<Entidad>`).

- **Obligatorio si es bidireccional:** El atributo `mappedBy`. Si la relación ya está definida en el otro lado con un `@ManyToOne`, aquí debes poner `@OneToMany(mappedBy = "nombreDelAtributoEnLaOtraClase")`. Esto le dice a JPA: *"No crees columnas nuevas, la clave foránea ya la gestiona la otra clase"*.

3. Relaciones @ManyToMany (Muchos a Muchos)

- **Obligatorio siempre:** La anotación `@ManyToMany` sobre una colección.
- **Obligatorio en el lado "inverso" (si es bidireccional):** Al igual que antes, debes usar `mappedBy = "..."` en una de las dos clases para que no se creen dos tablas intermedias distintas por accidente.
- *Práctica recomendada en el lado propietario:* Usar `@JoinTable` si necesitas controlar el nombre de la tabla intermedia generada y sus columnas (aunque si no lo pones, JPA se inventa los nombres automáticamente, como pasó en tu Test 4 con `challenge_announcement_recipients`).

4. Relaciones @OneToOne (Uno a Uno)

- **Obligatorio siempre:** La anotación `@OneToOne`.
- **Obligatorio en el lado "inverso":** El atributo `mappedBy`. Solo una de las dos tablas debe tener la clave foránea física; la otra usa `mappedBy` para leerla.

Si pone **`1`**: Es obligatorio que exista. Requiere `@NotNull` y `optional = false`.

Si pone **`0..1`**: Es opcional. No se pone `@NotNull`.

Si pone **`2..n`**: Exige una colección (Lista/Set) pero con un mínimo de elementos. Requiere `@NotNull` y `@Size(min = 2)`.

CONCEPTOS DE TEORÍA RELACIONADOS CON EL TEST 5

Arquitectura Software:

Lógica de Dominio: Representa las reglas de negocio puras y los comportamientos propios del problema que estás resolviendo (por ejemplo, calcular el precio de un servicio veterinario, comprobar si una mascota cumple las restricciones de edad, o validar invariantes de entidades).

Características principales:

- **Independiente de la tecnología:** No sabe nada de bases de datos, frameworks web (como Spring), JSON, protocolos HTTP ni interfaces de usuario.

Lógica de aplicación: Es la capa encargada de coordinar las tareas necesarias para que el sistema cumpla con un caso de uso específico. Actúa como un director de orquesta o intermediario entre el mundo exterior (controladores, peticiones HTTP) y el modelo de dominio. (CONTROLADORES).

Características principales:

- **Orquestación:** Se encarga de coordinar flujos como: "recibir una petición HTTP, comprobar la seguridad del usuario actual mediante el userService, llamar al repositorio para guardar una entidad, y gestionar las transacciones".
- **Dependiente de la tecnología:** Maneja aspectos técnicos como la serialización, la gestión de transacciones de bases de datos, la seguridad (permisos, tokens) y el manejo de excepciones web (como MethodArgumentNotValidException o AccessDeniedException).

*En la de aplicación solemos encontrar @Transactional

IMPORTANTE PONER @Service arriba de public class y @Autowired arriba del public.

Para el readOnly poner en los imports:

import org.springframework.transaction.annotation.Transactional;

Para el método save: @Transactional(rollbackFor = InvalidWinnerException.class)

Si al hacer la excepción en el save da error “Unhandled exception type UnfeasibleAttemptException “ o similar al usar una excepcion throw hay que declararla en la cabecera quedando de la siguiente manera:

```
@Transactional(rollbackFor = UnfeasibleAttemptException.class) public
ChessProblemAttempt save(ChessProblemAttempt m) throws UnfeasibleAttemptException {

        …

        }
```

IMPORTANTE AÑADIR @Service y los @Transactional

```
import org.springframework.transaction.annotation.Transactional;
```

CONCEPTOS DE TEORÍA RELACIONADOS CON EL TEST 6

El método va en la clase repositorio. Se pueden definir con @Query

Para hacer una consulta en el repositorio hay que extenderlo:

```
public interface ChessProblemRepository extends JpaRepository<ChessProblem, Integer> {


    @Query("SELECT cp FROM ChessProblem cp WHERE cp.difficultyLevel >= ?1 AND size(cp.board.pieces) < ?2")

        List<ChessProblem> findByDifficultyAndNumPieces(Integer difficultyLevel, Integer numPieces);
```

?1 = primer valor dado

?2= segundo valor dado

Ejemplo de consulta:
Obtener un listado de puntuaciones (Rating) dentro de una temporada ordenado por el ELO (primero el que más tenga) extraída del campo elo de la clase Rating (método getStandings(Season s)) .

```
        @Query("SELECT r FROM Season s JOIN s.ratings r WHERE s.id = :seasonId ORDER BY r.elo DESC")

List<Rating> findRatingsBySeasonIdOrderByEloDesc(@Param("seasonId") Integer seasonId);
```

CONCEPTOS DE TEORÍA RELACIONADOS CON EL TEST 7

Request Mapping annotations

- `@GetMapping`
- `@PostMapping`
- `@PutMapping`
- `@DeleteMapping`
- `@PatchMapping`

GET /owners/3/pets/23

```
@GetMapping("/owners/{ownerId}/pets/{petId}")
public Pet findPet(@PathVariable Long ownerId, @PathVariable Long petId) {
    // ...
}
```

ownerId = 3
petId = 23

- We can use exceptions to map HTTP error codes when something goes wrong

```
@ResponseStatus(HttpStatus.BAD_REQUEST)
public class BadRequestException extends RuntimeException {
}

@ResponseStatus(HttpStatus.NOT_FOUND)
public class ResourceNotFoundException extends RuntimeException {
}
```

```
@DeleteMapping("/{id}")
@ResponseStatus(HttpStatus.OK)
public void delete(@PathVariable("id") int id) {
    if (!petService.deletePet(id))
        throw new ResourceNotFoundException();
}
```



```
    @GetMapping

    public ResponseEntity<List<ChessProblemAttempt>> getAllAttempts() {

        List<ChessProblemAttempt> attempts =
matchService.getAttempts();

        return new ResponseEntity<>(attempts, HttpStatus.OK);

    }



    @GetMapping("/{id}")

    public ResponseEntity<ChessProblemAttempt>
getAttemptById(@PathVariable Integer id) {

        ChessProblemAttempt attempt = matchService.getMatchById(id)

        .orElseThrow(() -> new ResourceNotFoundException("Attempt not
found with id: " + id));
```

```
        return new ResponseEntity<>(attempt, HttpStatus.OK);

    }
```

HAY AVECES QUE HAY QUE AÑADIR AL SecurityConfiguration el .requestMatchers("/api/v1/attempts/**").hasAuthority(PLAYER) Cuando el enunciado nos diga que tenga que estar restringido a un usuario en especifico

CONCEPTOS DE TEORÍA RELACIONADO CON LOS TEST 9 Y 10

muestre los detalles de clasificación de un desafío:

```
export default function Challenge({ challengeId }) {

    const [ranking, setRanking] = useState([]);

    useEffect(() => {

        if (challengeId) {

            fetch(`/api/v1/challenges/${challengeId}/ranking`)

                .then((response) => {
```

```
                    if (response.ok) {
                        return response.json();
                    }
                    return [];
                })
                .then((data) => setRanking(data))
                    .catch((error) => { console.error("Error fetching
ranking:", error); });
        }
    }, [challengeId]);
```

```
import React, { useEffect, useState } from "react";
import { Table } from "reactstrap";

export default function AttemptsListing() {
    const [attempts, setAttempts] = useState([]);
    const [loading, setLoading] = useState(true);

    useEffect(() => {
        fetch("/api/v1/attempts")
            .then((response) => {
                if (response.ok) {
                    return response.json();
                }
```

```
                return [];

            })

            .then((data) => {

                setAttempts(data);

                setLoading(false);

            })

            .catch((error) => {

                console.error("Error fetching attempts:", error);

                setLoading(false);

            });



    }, []);



    if (loading) {

        return <h1>Loading...</h1>;

    }



     if (!attempts || attempts.length === 0) { //Hay que hacer lo de
loadign si te piden esto.

            return <h1>NO ATTEMPTS</h1>;

        }

     return … (tabla)

};
```

1. Cuándo usar `.map()` dentro de JSX (`<ul>` / `<table>`)

- **Pistas en el enunciado:**
  - *"Mostrar una lista de..."* o *"Mostrar una tabla con todos los..."*.
  - *"Recibe un array / colección de elementos por parámetro (props)"*.
  - *"Cada elemento debe mostrar..."*.
- **Regla fija en React:**
  Siempre que tengas un array en JavaScript y necesites convertirlo en elementos visuales (`<li>`, `<tr>`, `<div>`), **el único mecanismo estándar es `.map()`**.

```
{elementos && elementos.map(item => (

   <li key={item.id} ...>...</li>

))}
```

5. Resumen de lectura rápida en el examen

Si el enunciado o el test dice...	**Lo que debes escribir en el código**
"Muestre una lista/tabla de elementos"	{datos && datos.map(elem => (...))}
Estás dentro de un .map(...)	Poner siempre key={elem.id} en la etiqueta contenedora
El test comprueba .toHaveAttribute('id')	Añadir id={entidad-${elem.id}} a la etiqueta
El formato es "texto - número"	Escribir {elem.nombre} - {elem.valor} respetando los espacios

Crear una tabla:

```
return (
```

```
    <Table aria-label="attempts">
        <thead>
            <tr>
                <th>Name</th>
                <th>Start</th>
                <th>Finish</th>
                <th>Player</th>
                <th>Problem</th>
                <th>Dificulty</th>
            </tr>
        </thead>
        <tbody>
                {attempts.map((attempt) => (
                    <tr key={attempt.id}>
                        <td>{attempt.name}</td>
<td>{attempt.start ? new Date(attempt.start).toISOString() : ""}</td>
                                                <td>{attempt.finish ? new
Date(attempt.finish).toISOString() : ""}</td>
                            <td>{attempt.player ? attempt.player.username :
""}</td>
                              <td>{attempt.problem ? attempt.problem.name :
""}</td>
                                                    <td>{attempt.problem ?
attempt.problem.difficultyLevel : ""}</td>
                    </tr>
                ))}
        </tbody>
```

```
    </Table>


);
```

Si te dan arriba props:

```
export default function StandingsListing(props) {
```

```
    const { seasonId } = props;
    const [standings, setStandings] = useState([]);
    const [loading, setLoading] = useState(true);


    useEffect(() => {
        if (seasonId) {
            fetch(`/api/v1/seasons/${seasonId}/standings`)
                .then((response) => {
                    if (response.ok) {
                        return response.json();
                    }
                    return [];
                })
                .then((data) => {
                    setStandings(data);
```

```
                    setLoading(false);

                })

                .catch((error) => {

                    console.error("Error fetching standings:", error);

                    setLoading(false);

                });

        }

    }, [seasonId]);
```

COSAS IMPORTANTES:

En los repository no puede faltar el @Repository y el extends CrudRepository

En el service no puede faltar el @Service , el @Autowired (arriba del constructor) y los @Transactional

En el controlador no puede faltar @RestController, @PostMapping(“ruta”), @ResponseStatus(), @Autowired, que es del servicio. Además se usa @Requestbody @Valid, @GetMapping

> Si quieres poner una ruta base para todo el controlador se pone arriba @RequestMapping(“ruta”)
>
> El @Autowired se pone arriba del constructor.
>
> Importante el @PathVariable

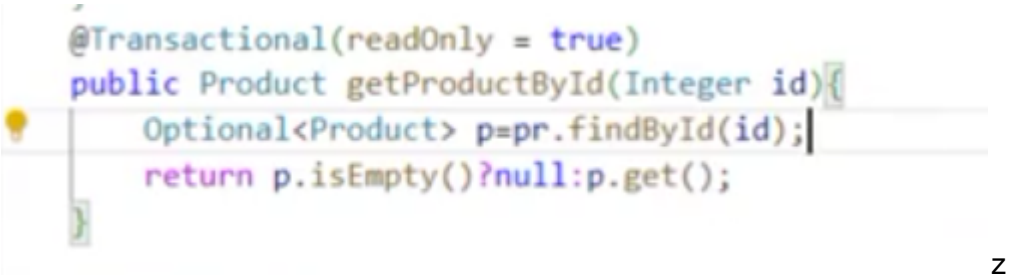
Los métodos en el controlador llevan el nombre de ResponseEntity

¿Cuándo debes usar cada una?

1. **Usa @NotBlank:**
    - **Exclusivamente en String** cuando el campo es obligatorio (nombres, apellidos, títulos, descripciones obligatorias, contraseñas, emails).

    - Evita que un usuario introduzca espacios en blanco para saltarse la validación.
2. **Usa @NotNull:**
    - En **fechas y tiempos** (LocalDate, LocalDateTime, etc.).
      MD
    - En **números y objetos envoltorio** (Integer, Double, BigDecimal, etc.).
      PDF
    - En **relaciones entre entidades** (User, Owner, ChessProblem, etc.).
    - En **enumerados** (PriceCode, Status, etc.).
3. **Usa @NotEmpty:**
    - En **colecciones, listas o sets** (List<...> , Set<...>) que deben tener al menos un elemento.

Si el getById devuelve un optional:



z

Excepción / Clase	Código HTTP	Nombre del Estado	Cuándo se produce / Uso principal
ResourceNotFoundException ZIP	**404**	NOT FOUND	Al buscar un recurso o entidad por ID y no existir en la base de datos.

AccessDeniedException ZIP	**403**	FORBIDDEN	El usuario no cuenta con el rol o autoridad necesaria (PLAYER, ADMIN, etc.) para acceder al endpoint.
ResourceNotOwnedException ZIP	**403**	FORBIDDEN	El usuario está autenticado pero intenta acceder o modificar un recurso que no le pertenece.
TokenRefreshException ZIP	**403**	FORBIDDEN	Fallo al renovar la sesión porque el token JWT expiró o no es válido.
UnfeasibleMatchException ZIP	**400**	BAD REQUEST	Violación de reglas de negocio al crear o guardar una partida (fechas incoherentes, ganador no participante, etc.).

WrongUserAssignmentException ZIP	**400**	BAD REQUEST	Asignación incorrecta o incompatible de usuarios dentro de la lógica del juego o torneo.
ErrorMessage *(DTO, no excepción)* ZIP	—	—	Objeto contenedor (payload) para estructurar el mensaje JSON de error que devuelve la API.

A los booleanos no hay que poner @NotNull porque toman valores false o true

Compilation error no puede dar (no corrige el examen)

```java
    @Transactional(rollbackFor = ResourceNotFoundException.class)
    public List<RankingEntry> getRanking(Integer courseId) {
        if (!repository.existsById(courseId)) {
            throw new ResourceNotFoundException(resourceName: "ChessCourse", fieldName: "id", courseId);
        }

        List<Object[]> results = repository.findStudentPoints(courseId);
        return results.stream()
                .map(row -> new RankingEntry((SchoolMember) row[0], (Long) row[1]))
                .toList();
    }
}
```

De una lista de objetos proporcionada por el repository, hacer una lista de algo y lanzar excepción en otro caso (Esto se hace en el service).

```java
@NotNull
@NotEmpty
@ManyToMany
List<SchoolMember> assignees;
```

Relacion MANYTOMANY si es una lista y es 1..n